ICT ACADEMY OF KERALA

Full Stack Application Development with ReactJS

Project: Library Management System

KANISHKA V M

SAHRDAYA COLLEGE OF ENGINEERING AND TECHNOLOGY, KODAKARA

25.10.2026

EXECUTIVE SUMMARY

This report describes the Library Management System, a full stack web application built during the Summer Internship in Full Stack Application Development with ReactJS. The application helps library staff keep track of books and members. A user can register and log in, add, edit and delete books, issue a book to a member, take it back, and view all registered members.

The project uses the MERN stack: MongoDB Atlas for the database, Express and Node.js for the server, and React (built with Vite) for the user interface. Passwords are hashed and login is handled with JSON Web Tokens. The server code is on GitHub and the React client is deployed on GitHub Pages. The main outcome is a working, deployed application and a good understanding of how the front end, back end and database fit together.

INTRODUCTION

Many small libraries still record books and borrowers on paper or in spreadsheets. This makes it hard to know which books are available, who has taken a book, and when it should come back. A simple web application can solve this by keeping all of this in one place that can be opened from any browser.

This project builds such an application as a learning exercise. It is a small system rather than a full commercial product. It covers the core library tasks (managing books, issuing and returning them, and viewing members) and leaves out larger features such as fines, reservations and barcode scanning.

The application follows the client-server model. The React client runs in the browser and sends requests to an Express API. The API checks the user's login token, reads or writes data in MongoDB Atlas through Mongoose, and sends the result back to the client.

OBJECTIVES

The objectives of the project were:

- To build a complete full stack application using React, Node.js, Express and MongoDB.
- To implement secure registration and login using hashed passwords and JSON Web Tokens.
- To create a REST API with routes for books, members and authentication.
- To build React screens for login, register, books and members.
- To connect the client to the API and keep the data stored in a cloud database.
- To use Git and GitHub for version control and to deploy the client online.
- To document the project with a README, an SRS and this report.

The skills I aimed to develop were React components and state, REST API design, database modelling with Mongoose, authentication, debugging, and deployment.

SCOPE AND DELIVERABLES

Scope

The system supports one type of user, a logged-in staff member. Any logged-in user can manage books and issue or return them. There are no separate admin and member roles in this version. Fines, due-date reminders, search filters and reservations are outside the scope.

Deliverables

- A Node.js and Express server with models, controllers, routes and authentication middleware.
- A React client with login, register, books and members screens.
- A MongoDB Atlas database holding users and books.
- A GitHub repository with a README, a screenshots folder and a .gitignore that keeps the private .env file out of the repository.
- The client deployed on GitHub Pages at https://kanishka0606-c.github.io/taskly/.
- A Software Requirements Specification (SRS).
- This project report.

METHODOLOGY

The project was built in small steps, testing each step before moving to the next. The work was done on a Windows computer using Visual Studio Code.

Tools and Technologies

Area	Technology
Frontend	React with Vite
Backend	Node.js v24, Express 5
Database	MongoDB Atlas with Mongoose 9
Security	bcryptjs for passwords, jsonwebtoken for login tokens
Other server packages	dotenv for settings, cors for allowed client address
Version control and hosting	Git, GitHub, GitHub Pages
Editor	Visual Studio Code

Approach

- The server was organised into models, controllers, routes and middleware, so each file has one job.
- Secrets such as the database connection string and the JWT secret were kept in a private .env file that is not uploaded to GitHub.
- Features were tested through the real browser screens after each step.
- The finished client was built with Vite and published through a GitHub Actions workflow.

PROJECT ACTIVITIES

- Project setup. Created the project folder, initialised the server with ES modules, and installed Express, Mongoose, dotenv, cors, bcryptjs and jsonwebtoken.
- Database connection. Created a MongoDB Atlas database with its own database user, and connected the server to it using the connection string from the .env file.
- User accounts. Built the User model and the register, login and current-user routes. Passwords are hashed before they are saved and are not returned in responses.

- Authentication middleware. Added a middleware that checks the Bearer token on every protected route.
- Book management. Built the Book model, controller and routes for listing, adding, editing and deleting books, plus issue and return actions.
- Members. Added a route to list all registered members.
- React client. Created the client with Vite and built the login, register, books and members screens, connected to the API.
- Testing. Tested the main actions in the browser: add, issue, return, edit, page refresh and delete.
- Version control and documentation. Published the code to GitHub and wrote the README.
- Deployment. Deployed the client to GitHub Pages using a GitHub Actions workflow.

Challenges and How They Were Solved

Challenge	Solution
Error saying a file "does not provide an export"	The file was empty or not saved; saving the file fixed it.
Error "port 5001 already in use"	An old server was still running; it was found and stopped before restarting.
Pasting into the terminal typed an odd character	Used right-click or Ctrl+Shift+V to paste instead.
Testing the API from the command line was confusing	Tested through the browser screens instead.
The deployed page showed the README instead of the app	The old GitHub Pages method was still publishing; it was switched off and the project's own workflow was run again.
Keeping secrets safe	The .env file was added to .gitignore and checked on GitHub to confirm it was not uploaded.

RESULTS AND FINDINGS

The project produced a working Library Management System that runs locally and is deployed online. Users can register, log in, manage the book list, issue and return books, and view members. Data is saved in MongoDB Atlas, so it stays after a page refresh.

Key Accomplishments

- A secure login system with hashed passwords and token-based access.
- A REST API with ten routes covering authentication, books and members.
- Clear separation of code into models, controllers, routes and middleware.
- A deployed client reachable from any browser.
- A repository that does not contain the .env file or node_modules.

API Summary

Method	Route	Purpose
POST	/api/auth/register	Create an account
POST	/api/auth/login	Log in and get a token
GET	/api/auth/me	Get the logged-in user
GET	/api/books	List all books
POST	/api/books	Add a book
PUT	/api/books/:id	Edit a book
PUT	/api/books/:id/issue	Issue a book to a member
PUT	/api/books/:id/return	Return a book
DELETE	/api/books/:id	Delete a book
GET	/api/members	List all members

Limitations

- There is only one user role, so every logged-in user has full access.
- There are no due dates, fines or search filters.
- The deployed client depends on the server and database being available.

CONCLUSION

The project met its main objectives. I built and deployed a complete MERN application with authentication, a REST API, a React interface and a cloud database. Along the way I learned how the parts of a full stack application communicate, how to protect passwords and secrets, how to read and fix common errors, and how to publish work using Git and GitHub.

The system can be improved in future by adding separate admin and member roles, due dates and fines for late returns, search and filtering, and email reminders for overdue books.

APPENDIX

- A. Repository: https://github.com/kanishka0606-c/taskly
- B. Live site: https://kanishka0606-c.github.io/taskly/
- C. Software Requirements Specification (separate document)
- D. Folder structure: the client folder holds the React app, and the server folder holds the Express API (models, controllers, routes, middleware).
- E. Environment variables used by the server: PORT, MONGO_URI, JWT_SECRET and CLIENT_ORIGIN (values are private and not included).